Ders 2 Genişletilmiş Notu: Eğitmenli Öğrenme ve Karar Ağaçları: Entropi, Bilgi Kazancı ve ID3

Ders 2 Genişletilmiş Notu: Eğitmenli Öğrenme ve Karar Ağaçları: Entropi, Bilgi Kazancı ve ID3

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Derse Giriş ve Lisansüstü Ders Seçimi
- 2. Eğitmenli Öğrenmenin Yapısı
- 3. Regresyon ve Sınıflandırma
- 4. Eğitmenli Öğrenmenin İki Aşaması
- 5. Veri Tipleri ve Özelliklerin Ayırt Ediciliği
- 6. Karar Sınırları
- 7. Karar Ağaçlarının Mantığı ve Yapısı
- 8. En İyi Özellik: Saflık ve Safsızlık
- 9. Entropi
- 10. Bilgi Kazancı
- 11. PlayTennis Örneğiyle Adım Adım Ağaç Kurma
- 12. ID3 Algoritması ve Occam'ın Usturası
- 13. Aşırı Öğrenme (Overfitting)
- 14. Budama (Pruning)
- 15. Karar Ağaçlarının Güçlü ve Zayıf Yanları

1. Derse Giriş ve Lisansüstü Ders Seçimi

Dersin ilk yaklaşık 20 dakikasında öğrenciler kameralarını açıp kendilerini tanıttı. Hoca bunu, derslerin uzun süre çevrim içi (online) devam edeceği düşüncesiyle, öğrencilerin birbirini tanıyabilmesi için istedi. Bu bölüm akademik içerik taşımadığı için nota alınmamıştır.

Tanışmanın sonunda bir öğrencinin "umarım yapay zeka zehirlenmesine uğramam" sözü üzerine hoca, lisansüstü ders seçimiyle ilgili bir tavsiye verdi. Hoca yapay zekaya lisans 4. sınıfta, kalp hastalıklarının teşhisi için bir uzman sistem (expert system) yazarak başlamış. Yüksek lisans ve doktorasını görüntü işleme ağırlıklı yapmış, ama makine öğrenmesi ve yapay zeka bu alanlarla iç içe olduğu için onları da hep kullanmış.

Hocanın tavsiyesi şöyle:

- Lisansüstü eğitim, lisansta görülen konularda uzmanlaşmayı ve derinleşmeyi sağlar. Bilgisayar mühendisliğinin genel konularında ne kadar bilgi sahibi olursanız ileride o kadar faydasını görürsünüz.
- Toplam 7 dersin 3-4 tanesi uzmanlaşmak istediğiniz alandan olsun. Örneğin makine öğrenmesi ile birlikte, görüntü işleyecekseniz görüntüyle ilgili bir ders, doğal dil işleyecekseniz doğal dil işlemeyle ilgili bir ders.
- Kalan derslerin en az 1-3 tanesini bilgisayar mühendisliğinin başka alanlarından, kendinizi zayıf hissettiğiniz konulardan seçin. Yolun nereye gideceği kestirilemez. Örneğin güvenlik (security) çok önemli hale gelirse, yapay zeka ile güvenliği birlikte kullanan projeler yapabilirsiniz.

- Birbirine çok benzeyen dersleri birlikte almayın. Makine öğrenmesi, veri madenciliği ve yapay zeka derslerinin konularının yaklaşık %40'ı ortaktır. Bunlardan birini alıp eksikleriniz kendiniz tamamlayabilirsiniz.

2. Eğitmenli Öğrenmenin Yapısı

Geçen derste üç ana öğrenme yöntemi tanıtılmıştı:

- Eğitmenli öğrenme (supervised learning).
- Eğitmensiz öğrenme (unsupervised learning).
- Pekiştirmeli öğrenme (reinforcement learning): ödül-ceza ile öğrenme.

Bu dersten itibaren bir süre eğitmenli öğrenmenin önemli konuları ve modelleri işlenecek. Eğitmensiz ve pekiştirmeli öğrenme daha sonra gelecek.

2.1 Veri kümesi her şeyin başı

Eğitmenli öğrenmede elimizde N örnekten oluşan bir veri kümesi (dataset) vardır. Hocaya göre eğitmenli öğrenmede başrolü veri kümesi oynar. İyi bir veri kümesi yoksa düzgün bir sistem hiçbir zaman kurulamaz.

> **Hocanın vurgusu:** Eğitmenli öğrenmeye dayalı bir projeye (yüksek lisans tezi, doktora tezi, Ar-Ge projesi) başlamadan önce ilk iş, böyle bir veri kümesinin var olup olmadığını ve elde etmenin kolay olup olmadığını araştırmaktır. Veri yoksa, proje başlayana kadar toplanabilecek mi diye bakın. Proje başladıktan sonra veri toplamaya çok güvenmeyin.

Hoca bu konunun yıllarca atlandığını, "önce projeye başlayalım, veri kümesini sonra oluştururuz" görüşünün yaygın olduğunu söyledi. Örnek olarak TÜBİTAK desteğine başvurulmuş, uçak yolcularına tavsiye sistemi tasarlayan bir projeyi anlattı. Proje IATA ile birlikte yürütülecekti. Ancak başlangıcı pandemiye denk geldi. Ellerinde hazır bir veri kümesi yoktu ve uluslararası uçuşlar kısıtlandığı için veri toplanamadı. Bir yıl içinde bitmesi gereken proje bu yüzden yürüyemedi.

2.2 Verinin biçimi

Veri kümesindeki her örnek iki bilgiden oluşur:

- x_i: i. örneğin özellikleri (giriş).
- y_i: aynı örneğin çıkışı. Buna **etiket (label)** denir. Sınıflandırmada etiket, örneğin ait olduğu sınıftır (A sınıfı, B sınıfı gibi).

Bu durumda veri kümesi şöyle gösterilir:

$$D = \{(x_i, y_i)\}_{i=1}^{N}$$

Eğitmenli öğrenmenin amacı, x girişlerinden y çıkışına götüren bir f fonksiyonu bulmaktır:

$$y \approx f(x)$$

Bu fonksiyon, verilen x girişleri için gerçek y çıkışına (sizin etiketle verdiğiniz çıkışa) en yakın sonucu vermelidir. Yani hatası en az olmalıdır. Böylece sınıfı bilinmeyen yeni bir x geldiğinde, onun hangi sınıfa ait olduğu en yüksek başarıyla tahmin edilebilir.

2.3 Başarıyı ölçmek

> **Hocanın vurgusu:** Bir model kurmak yetmez, başarısı ölçülmelidir. "Bu model yüz tanıyor" demek yeterli değildir; önemli olan hangi başarıyla tanıdığıdır.

Hoca bunu deterministik problemlerle karşılaştırdı. “10 sayıyı küçükten büyüğe sıralayın” problemi deterministiktir. Verilen 10 sayı için doğru sıralama her zaman aynıdır. Program çalışıyorsa her girdi için doğru sonucu verir. Yapay zeka problemleri ise deterministik değildir. Genellikle %100 başarı yoktur. Bu yüzden hedef, en başarılı öğrenme modelini kurmak ve kurduktan sonra başarısını ölçüp yeterli olup olmadığına karar vermektir.

3. Regresyon ve Sınıflandırma

Eğitmenli öğrenmede iki ana öğrenme biçimi vardır: regresyon ve sınıflandırma. Aralarındaki fark çıkışın türündedir.

3.1 Regresyon (regression)

Giriş sürekli ya da ayrık olabilir, **çıkış süreklidir**. Sonuçta bir sınıf yoktur, bir değer tahmin edilir.

- **Mısır hasılatı örneği:** Bir bölgede 12 ay boyunca aylık yağış miktarı ve buna karşılık gelen mısır hasılatı veriliyor. Örneğin bir ay 3 ton yağış gelmiş, 400 kg mısır olmuş; başka bir ay 2 ton yağış gelmiş, 500 kg mısır olmuş. Soru: “Aralık’ta 1 ton yağış olursa kaç kilo mısır alırım?” Çıkış bir sayıdır, bu bir regresyon problemidir. Aynı soru “hasılat az mı olur çok mu?” diye sorulsaydı, iki sınıflı bir sınıflandırma problemi olurdu.
- **Ev fiyatı örneği:** Bir bölgedeki evlerin metrekaresi, oda ve salon sayısı, yaşı gibi özellikleri ve fiyatları eğitim bilgisi olarak veriliyor. Soru: “3 oda 1 salon, 140 m^2, 10 yaşında bir ev kaç liradır?” Cevap bir fiyattır, regresyon problemidir.
- **Sıcaklık tahmini:** Verilen değerlere göre sıcaklığın ne olacağını tahmin etmek.

3.2 Sınıflandırma (classification)

Giriş sürekli ya da ayrık olabilir, **çıkış ayrık bir sınıftır**.

- Aynı ev bilgileriyle “bu ev ucuz mu pahalı mı?” sorusu bir sınıflandırma problemidir.
- **Kredi kartı sahtekarlığı:** Harcamanın tutarı, nerede yapıldığı, kartın tipi giriş bilgileridir. Çıkışta iki sınıf vardır: sahte harcama ya da normal harcama.
- **Konuşma tanıma:** Giriş tek boyutlu, sürekli bir ses sinyalidir. Çıkışta hangi kelimenin söylendiği bulunur (“kırmızı”, “yeşil”, “defter” gibi). Her kelime bir sınıftır.
- **Konuşmacı tanıma:** Sesten kimin konuştuğunu bulmak. Her kişi bir sınıftır.

3.3 Hangisini seçmeli?

> **Hocanın vurgusu:** İlk adım, problemin regresyonla mı sınıflandırmayla mı çözülmesi gerektiğine karar vermektir.

Ev fiyatı probleminde “ucuz / pahalı” demek genellikle ihtiyacı karşılamaz; fiyatın kendisi gerekir. Bu yüzden orada regresyon daha mantıklıdır. Sesten kimlik tanımada ise her kişi bir sınıftır; sınıflandırma modeli kullanılır.

Hoca bir uyarı da ekledi: **Lojistik regresyon (logistic regression)** adında “regresyon” geçse de buradaki regresyondan farklı değerlendirilir. İleriki haftalarda anlatılacak.

> **Ek bilgi (derste anlatılmadı):** Lojistik regresyon, adına rağmen bir sınıflandırma yöntemidir. Çıkışı bir sınıfa ait olma olasılığıdır; bu olasılık bir eşikle karşılaştırılarak sınıf kararı verilir.

4. Eğitmenli Öğrenmenin İki Aşaması

Eğitmenli öğrenme iki ana adımdan oluşur.

1. **Model eğitimi:** Girişleri x, çıkışları y olan D örnekleriyle bir model eğitilir. Hocanın saydığı model örnekleri: k-en yakın komşu (k-nearest neighbor, k-NN), karar ağacı (decision tree), destek vektör makineleri (support vector machines, SVM), derin öğrenme (deep learning). Eğitim sonunda elimizde, bir örneğin özellikleri verildiğinde çıkışını söyleyebilen

bir model olur. Kredi kartı örneğinde özellikler, harcamanın nerede ve ne kadar yapıldığı gibi bilgilerdir.
2. **Tahmin:** Sınıfı bilinmeyen (regresyonda değeri bilinmeyen, örneğin fiyatı bilinmeyen bir ev) yeni bir x örneği gelir. Bu örnek modelden geçirilir ve y çıkışı tahmin edilir.

5. Veri Tipleri ve Özelliklerin Ayırt Ediciliği

Bu kısım geçen dersin tekrarıdır. Özelliklerin tipi üç gruba ayrılır.

- **Ordinal:** Aralarında ilişki kurulabilen, ölçülebilen, sıralanabilen sayılar. Yaş, boy, sıcaklık gibi. Bir kişinin 30 kg olmasıyla 33 kg olması arasındaki fark ölçülebilir ve anlamlıdır. 32 kg, 30 kg'dan 2 kg fazladır; aralarında bir mesafe vardır.
- **Kategorik (nominal):** Farklı değerler alır ama değerlerin birbirine göre ölçülebilir bir üstünlüğü ya da mesafesi yoktur. Örneğin kartın kullanıldığı şehir: İstanbul, Ankara, İzmir. Bu şehirlerin benzerliğini ölçmek için sayısal bir mesafe hesaplanmaz.
- **Anlamsız (useless):** Kimlik numarası, ad soyad gibi kişiye özel, sınıflandırmada anlam taşımayan bilgiler. Öğrenmede kullanılmamalı, en baştan dikkatle elenmelidir.

5.1 İyi özellik nedir?

Hangi özelliklerin kullanıldığı, sınıflandırma başarısını doğrudan etkiler. Özellik seçimi (feature selection) ileride ayrı bir derste anlatılacak. Şimdilik temel ilke şudur:

> **Hocanın vurgusu:** Özellikler aynı sınıftaki örneklerin birbirine yakın olmasını, farklı sınıflardaki örneklerin birbirinden uzak olmasını sağlamalıdır.

Örnek: Yüz tanıma probleminde göz rengi ayırt edici bir özellik değildir. Türkiye'de çoğu kişinin gözü kahverengidir. "Bu Elif mi, Ayşe mi?" sorusuna göz rengi cevap veremez. Özellik seçiminde önemli olan, sınıflar arasındaki benzerliği azaltan, sınıflar arasındaki farkı artıran özellikleri seçmektir.

6. Karar Sınırları

Bu derste sınıflandırma ve bir sınıflandırma yöntemi olarak karar ağaçları anlatılıyor. Regresyon daha sonra işlenecek.

Sınıfları birbirinden ayırmak için bir **karar sınırına (decision boundary)** ihtiyaç vardır. Eğitmenli öğrenmedeki bütün modeller, yapılarına bağlı olarak bir şekilde karar sınırı oluşturur. Sınıfı bilinmeyen bir örnek sınırın hangi tarafında kalıyorsa o sınıfa atanır. Eğitmenli öğrenmede olay, iyi bir karar sınırı oluşturmak üzerine kuruludur. Hedef, sınıflandırma başarısı yüksek olacak şekilde en iyi karar sınırını çizmektir.

Slayttaki iki örnek:

- **Doğrusal (linear) karar sınırı:** İki özellik var: öğrencilerin iki sınavdan aldığı notlar. Bir sınıf kabul edilen (admitted) öğrenciler, öbür sınıf kabul edilmeyenler. En iyi doğrusal sınır iki grubun arasından geçiyor. Ama sınırın iki yanında da diğer sınıftan birkaç örnek kalıyor.
- **Doğrusal olmayan (non-linear) karar sınırı:** Yine iki özellik var. Yeşil örnekler içeride, kırmızı örnekler dışarıda toplanmış. Sınır bir eğri.

İki örnekte de %100 başarı yoktur. Kırmızı bölgede birkaç yeşil, yeşil bölgede birkaç kırmızı örnek kalır.

> **Hocanın vurgusu:** Bütün eğitim örneklerini doğru sınıflandıracak bir sınır çizmeye kalkarsanız bu ezberlemeye gider. Bu istenen bir şey değildir. Genel bir çözüm üretmek gerekir.

Doğrusal olmayan sınır çizmek her zaman zordur. Problemler bu örnekler kadar düzgün olmayabilir. Doğrusal olarak ayrılamayan karmaşık problemlerde sınırı belirlemek için iyi bir matematiksel model kurmak gerekir.

7. Karar Ağaçlarının Mantığı ve Yapısı

7.1 Kararlarla sınır çizmek

Karar ağacı, doğrusal olmayan bir karar sınırını matematiksel bir model kurmadan çizer. Bunu "eğer-ise" (if) kontrolleriyle yapar: "Şöyleyse sınırı buradan geçir, böyleyse şuradan geçir."

Slayttaki örnekte iki özellik (x_1, x_2) ve iki sınıf (0 ve 1) vardır. Kurulan ağacın altyazıda anlatılan kısmı şöyledir:

- $x_2 < 3$ ve $x_1 < 4$ ise: sınıf 0.
- $x_2 < 3$ ve $x_1 \geq 4$ ise: sınıf 1.
- $x_2 \geq 3$, $x_1 < 3$ ve $x_2 < 4$ ise: sınıf 0.

```
x2 < 3 ?
├─ evet:  x1 < 4 ?
│         ├─ evet  -> sınıf 0
│         └─ hayır -> sınıf 1
└─ hayır: x1 < 3 ?
          ├─ evet: x2 < 4 ?
          │        ├─ evet -> sınıf 0
          │        └─ ...
          └─ ...
```

Her koşul, özellik uzayını eksenlerden birine dik bir çizgiyle böler. Örneğin $x_1 < 4$ ve $x_2 < 3$ koşulları birlikte sol alttaki dikdörtgen bölgeyi tanımlar; bu bölgede sıfırlar toplanmıştır. Böylece karar ağacı özellik uzayını sınırlarla bölgelere ayırır ve her bölgede yalnız bir sınıfa ait örnekler kalır. Ortaya çıkan toplam sınır doğrusal değildir ama esnektir.

Bir öğrenci karar ağacının mantığını tam anlamadığını söyledi. Hoca, ağaçların kuruluşunu birkaç slayt sonra örnekle anlatacağını, sonra da anlaşılmayan yer kalırsa tekrar edeceklerini söyledi.

7.2 Nominal özellikler

> **Hocanın vurgusu:** Karar ağaçlarını birçok öğrenme modelinden farklı ve avantajlı kılan özelliklerden biri, nominal değerli özelliklerin doğrudan kullanılabilmesidir.

Örneğin "renk kırmızı mı, yeşil mi, mavi mi?" gibi bir özellik ağaçta doğrudan bir düğüm olabilir. Sayıya çevirmeye gerek yoktur.

7.3 Düğüm, dal, yaprak

Hoca yapıyı ilerideki PlayTennis örneğinin ağacı üzerinden anlattı.

- **Düğüm (node):** Bir özelliktir. Örneğin Outlook (hava durumu) bir düğüm, Humidity (nem) başka bir düğümdür.
- **Dal (branch):** Düğümdeki özelliğin aldığı bir değerdir. Outlook düğümünün üç dalı vardır: Sunny, Overcast, Rain.
- **Yaprak (leaf):** Sınıftır. Ağaç, bir daldaki bütün örnekler aynı sınıfa ait olunca o dalda biter.

Örneğin Outlook = Sunny dalında ağaç Humidity'ye bakmaya devam eder. Humidity = High ise sınıf No, Humidity = Normal ise sınıf Yes'tir. Outlook Sunny iken Humidity High olan günlerin hiçbirinde tenis oynanmamıştır, bu yüzden dal orada biter. Eğer Humidity High iken bazen oynanmış, bazen oynanmamış olsaydı, ağaç bütün örnekler aynı sınıfa düşene kadar aşağı doğru devam ederdi.

7.4 Sürekli özellikler

Özellik değerlerinin Sunny, High, Normal gibi kesin kategoriler olması gerekmez. Sıcaklık gibi sürekli bir özellik için 15, 16, 17, 18 dereceyi ayrı ayrı değerlendirmek yerine aralık kullanılabilir. Örneğin 20 dereceden büyüklere true, küçüklere false denebilir. Eşik değerinin (burada 20) nasıl belirleneceği ayrı bir konudur ve sonra anlatılacak.

7.5 Ağaçtan kural çıkarma

Kurulan bir karar ağacı bir kural grubuyla gösterilebilir. Bilgisayarda karar ağacı aslında kural hâlinde kullanılır. PlayTennis ağacının kuralları:

```
EĞER (Outlook = Sunny VE Humidity = Normal)
   VEYA (Outlook = Overcast)
   VEYA (Outlook = Rain VE Wind = Weak)        İSE PlayTennis = Yes

EĞER (Outlook = Sunny VE Humidity = High)
   VEYA (Outlook = Rain VE Wind = Strong)      İSE PlayTennis = No
```

Yes kuralındaki üç koşuldan biri sağlandığında tenis oynanmıştır.

8. En İyi Özellik: Saflık ve Safsızlık

8.1 PlayTennis problemi

Hoca ağaç kurmayı Tom Mitchell'ın "Machine Learning" kitabındaki, karar ağaçları için çok kullanılan bir örnekle anlattı. Hikaye şöyle: Utangaç bir genç, bir kızı birlikte tenis oynamaya davet etmek istiyor ama çekiniyor. 14 gün boyunca her günün hava koşullarını ve kızın o gün tenis oynayıp oynamadığını not alıyor. Örneğin birinci gün hava sıcak, nem yüksek, rüzgar zayıf ve tenis oynanmamış. İkinci gün hava sıcak, nem yüksek, rüzgar kuvvetli ve yine oynanmamış. 15. gün, o günün koşullarına bakarak kızın tenis oynayıp oynamayacağını tahmin edecek ve oynayacaksa davet edecek.

Problemde 4 özellik ve 2 sınıf vardır:

Özellik	Değerler
Outlook (hava durumu)	Sunny, Overcast, Rain
Temperature (sıcaklık)	Hot, Mild, Cool
Humidity (nem)	High, Normal
Wind (rüzgar)	Weak, Strong
Sınıf: PlayTennis	Yes, No

14 günlük eğitim verisi (Mitchell'ın kitabındaki standart tablo; hoca slayttan gösteriyor, altyazıda gün gün okunmuyor):

Gün	Outlook	Temperature	Humidity	Wind	PlayTennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Aşağıdaki bütün sayısal hesaplar bu tabloyla tutarlıdır ve python ile doğrulanmıştır.

8.2 İyi bir ağaç nasıl olmalı?

Bu 14 örnek için birçok farklı karar ağacı kurulabilir. Hangisini istediğimizi düşünürsek iki şart çıkar:

1. Sınıflandırma başarısı yüksek olmalı.
2. Ağaç mümkün olduğunca kısa olmalı. Yani dallanma az, yapraklara olan mesafe (ağacın yüksekliği) düşük olmalı.

   Hocanın vurgusu: Bunu sağlamanın yolu, en iyi (en ayırt edici) özelliği köke yerleştirmek ve oradan devam etmektir.

Ağacı rastgele de kurabilirdik. Örneğin rastgele nemi seçip köke koyar, nemin her değeri için altına başka bir rastgele özellik koyar, bütün örnekler aynı sınıfa düşene kadar böyle devam ederdik. Bu bir ağaç verir ama etkili (efektif) bir çözüm değildir.

8.3 Saf bölünme

En iyi özellik, örnekleri olabildiğince **saf (pure)** gruplara bölen özelliktir. Saf grup, bütün örnekleri aynı sınıfa ait olan gruptur.

Slayttaki örnekte iki sınıflı (yeşil ve kırmızı) bir başvuru verisi için iki aday özellik karşılaştırılıyor:

- **"Ağırlık / bakiye 50K'dan büyük mü?"** özelliği: 50K'dan küçük tarafta çoğunluk yeşil, ama arada bir kırmızı var. Büyük tarafta çoğunluk kırmızı, ama birkaç yeşil var. İki tarafta da bir kontrol daha yapmak gerekecek.
- **"Çalışıyor mu (employed)?"** özelliği: Çalışanlar tarafında yalnız bir sınıf, çalışmayanlar tarafında yalnız öbür sınıf var.

Hoca öğrencilere hangisinin köke daha uygun olduğunu sordu; cevap sağdaki (employed) oldu. Çünkü "employed" köke konunca ağaç hemen biter: true ise bir sınıf, false ise öbür sınıf. Daha aşağı inmeye gerek yoktur. Daha küçük, yüksekliği daha az, daha az karmaşık bir ağaç elde edilir.

8.4 Safsızlık (impurity)

Safsızlık, bir gruptaki örneklerin farklı sınıflardan ne kadar karışık olduğunu anlatır. Slaytta dört grup sırayla gösterildi:

- Bütün örnekleri aynı sınıfta olan bir grup: tamamen saf.
- Çoğunluğu bir sınıfta, az sayıda örneği öbür sınıfta olan bir grup: saflığa yakın.
- İki sınıftan yaklaşık yarı yarıya örnek içeren gruplar: çok karışık, saf değil.

Bilgi kazancında amaç, eğitim kümesindeki en iyi özelliği seçip köke yerleştirmektir. En iyi özellik seçilirse baştan iyi bir bölünme sağlanmış olur. Sonra yalnız aynı sınıfta olmayan örneklerin kaldığı dallar için aşağı doğru devam edilir.

Bilgi kazancı (information gain), özellik vektöründeki her özelliğin sınıfları ayırmada ne kadar önemli olduğunu ölçmek için kullanılır. İki tarafında da bir sınıfın belirgin çoğunlukta olduğu bir bölünme, aşağı doğru daha az devam edecektir. İki tarafı da yarı yarıya karışık kalan bir bölünmenin bilgi kazancı düşüktür ve ağacı çok daha fazla büyütmek gerekebilir.

9. Entropi

Bilgi kazancını hesaplamak için önce **entropi (entropy)** kavramı gerekir.

Tanım: Entropi, bir kümedeki örnekleri kodlamak için gereken en az bit sayısıdır.

c tane sınıf varken, bir örneğin i. sınıfa ait olma olasılığı p_i ise:

$$E(S) = -\sum_{i=1}^{c} p_i \log_2 p_i$$

Burada p_i, S kümesindeki örneklerden i. sınıfa ait olanların oranıdır.

İki sınıflı (artı / eksi) durumda formül şuna dönüşür:

$$E(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

p_+: artı sınıfındaki örneklerin oranı, p_-: eksi sınıfındaki örneklerin oranı.

9.1 Entropinin aldığı değerler

İki sınıfta entropi 0 ile 1 arasında değişir. Slayttaki grafikte yatay eksen p_+, dikey eksen entropidir:

- Artı sınıfında hiç örnek yoksa ($p_+ = 0$, bütün örnekler eksi sınıfta): $E = 0$.
- Bütün örnekler artı sınıfındaysa ($p_+ = 1$): $E = 0$.
- Örneklerin %50'si artı, %50'si eksi sınıfındaysa ($p_+ = 0.5$): $E = 1$.

Yani bütün örnekler aynı sınıftaysa entropi 0'dır. Örnekler sınıflara eşit dağılmışsa entropi en yüksektir. Çok saf bir grubun entropisi düşük, saflığı az olan bir grubun entropisi yüksektir.

> **Ek bilgi (derste anlatılmadı):** $0 \cdot \log_2 0$ terimi tanımsız görünür, ama limiti 0'dır; hesapta 0 kabul edilir. c sınıflı bir problemde entropinin en büyük değeri $\log_2 c$'dir; iki sınıfta bu 1 eder.

Hoca, entropinin c sınıflı genel formülünü bir öğrencinin sorusu üzerine dersin ilerleyen kısmında tekrar açıkladı. İki sınıflı gösterimi, grafik üzerinde anlaşılır olduğu için tercih ettiğini söyledi. Sınıf sayısı ikiden fazla olunca formül değişmez; bütün sınıflar için $-p_i \log_2 p_i$ terimleri toplanır.

9.2 Örnek 1: Toplam entropi

Verilen: 64 örnek. 29'u artı sınıfında, 35'i eksi sınıfında.

İşlem:

$$E(S) = -\frac{29}{64} \log_2 \frac{29}{64} - \frac{35}{64} \log_2 \frac{35}{64}$$

$\frac{29}{64} \approx 0.453$, $\frac{35}{64} \approx 0.547$. Bu oranlarla:

$$E(S) \approx 0.453 \cdot 1.142 + 0.547 \cdot 0.871 \approx 0.99$$

Sonuç: $E(S) \approx 0.99$. İki sınıfta yaklaşık eşit sayıda örnek olduğu için entropi 1'e yakındır.

9.3 Sınıf dengesi

Hoca bu örnekten yola çıkarak veri kümesi hazırlamayla ilgili bir uyarı yaptı.

> **Hocanın vurgusu:** Veri kümesini oluştururken, modelin iyi öğrenebilmesi için mümkün olduğunca her sınıfta eşit sayıda örnek olmasını sağlamaya çalışın.

İki sınıf varsa, toplanabilen örnek sayısına göre %50-%50 ya da %40-%60 gibi, dengeyi çok bozmayan bir oran hedeflenmelidir. Örnekler dengesiz olduğunda, örneği az olan sınıfın karakteristik özelliklerini öğrenmek zorlaşır. Dengesizliğin oranı da başarıyı doğrudan etkiler. Örneğin bir sınıfta 1, öbüründe 40 örnek varsa bu çok dengesiz bir durumdur ve sınıflandırma başarısı düşebilir.

10. Bilgi Kazancı

Tanım: Bilgi kazancı, bir A özelliğinin S örnek topluluğu için beklenen kazancıdır:

$$Gain(S, A) = E(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} E(S_v)$$

- $Values(A)$: A özelliğinin alabildiği değerler.
- S_v: A özelliğinin değeri v olan örnekler. Örneğin A rüzgar ise, S_{Strong} rüzgarın kuvvetli olduğu örneklerdir.
- $\frac{|S_v|}{|S|}$: bu örneklerin bütün örneklere oranı.
- $E(S_v)$: yalnız bu örnekler için hesaplanan entropi.

Hesap şöyle yapılır: Özelliğin her değeri için, o değere sahip örneklerin entropisi örnek oranıyla çarpılır ve toplanır. Bu toplam, bütün örneklerin entropisinden çıkarılır. Sonuç, o özelliği kullanarak bölmekle kazanılan bilgidir.

Her özellik için kazanç hesaplanır (nemin kazancı, sıcaklığın kazancı, ...). **Kazancı en yüksek özellik en iyi özelliktir ve köke yerleştirilir.**

10.1 Örnek 2: A1 mi A2 mi?

Verilen: Örnek 1'deki 64 örnek ([29+, 35−], $E(S) \approx 0.99$). İki aday özellik var: $A1$ ve $A2$. İkisinin değerleri de true ve false.

Özellik	Değer	Örnekler	Örnek sayısı
A1	true	[21+, 5−]	26
A1	false	[8+, 30−]	38
A2	true	[18+, 33−]	51
A2	false	[11+, 2−]	13

Adım 1: A1'in dallarının entropisi.

$$E(A1 = true) = -\frac{21}{26}\log_2\frac{21}{26} - \frac{5}{26}\log_2\frac{5}{26} \approx 0.71$$

$$E(A1 = false) = -\frac{8}{38}\log_2\frac{8}{38} - \frac{30}{38}\log_2\frac{30}{38} \approx 0.74$$

Adım 2: A1'in kazancı. A1 true olan 26 örnek, false olan 38 örnek var.

$$Gain(S, A1) = 0.99 - \frac{26}{64}(0.71) - \frac{38}{64}(0.74) \approx 0.27$$

Adım 3: A2 için aynı işlem (ara adım eklendi). Hoca A2 için yalnız sonucu verdi.

$$E(A2 = true) = E([18+, 33-]) \approx 0.94, \qquad E(A2 = false) = E([11+, 2-]) \approx 0.62$$

$$Gain(S, A2) = 0.99 - \frac{51}{64}(0.94) - \frac{13}{64}(0.62) \approx 0.12$$

Sonuç: $Gain(S, A1) \approx 0.27 > Gain(S, A2) \approx 0.12$. **A1 köke yerleştirilir.**

Sezgisel yorum: A1 true iken örneklerin çoğunluğu artı sınıfında, az miktarda eksi var. A1 false iken çoğunluk eksi sınıfında, az miktarda artı var. Yani A1 sınıfları oldukça başarılı biçimde bölüyor. A2 true dalında ise 18'e 33 var; bu taraf neredeyse yarı yarıya karışık. Hoca

iki kazanç arasında çok büyük bir fark olmadığını (biri 0.8, öbürü 0.03 gibi değil) ama gözle bakınca da A1 tarafının daha umut verici göründüğünü söyledi.

Soru: Burada toplam entropi hangisi oluyordu? **Cevap:** 0.99. İki sınıf ve 64 örnek var; 29'u artı, 35'i eksi. Bu, bütün örneklerin entropisidir. A1 true iken 26 örnek var, 21'i artı, 5'i eksi; A1 false iken 38 örnek var, 8'i artı, 30'u eksi. Bu iki grubun entropileri ayrı ayrı (0.71 ve 0.74) hesaplanır ve örnek oranlarıyla (26/64 ve 38/64) çarpılarak toplam entropiden çıkarılır.

Öğrenci ardından "toplam entropi hiç değişmeyecek, bütün özellikler için hep aynı olacak, değil mi?" diye sordu. Hoca bunu onayladı: Toplam entropi bütün örneklerden hesaplanır, hangi özellik incelenirse incelensin aynıdır. Kazanç, bu toplam entropiden, özelliğe göre bölündükten sonra kalan (ağırlıklı) entropinin çıkarılmasıyla bulunur.

Soru: A1 true dalındaki 26 örneğin hepsinin artı, false dalındaki 38 örneğin hepsinin eksi olması en iyi ayrılma durumu mu? **Cevap:** Evet. A1 true iken bütün örnekler tek bir sınıfta olsaydı problem tek adımda çözülmüş olurdu. Gerçekte dallar karışıktır, bu yüzden ağaç aşağı doğru devam eder. Burada anlaşılmaya çalışılan, hangi özelliğin örnekleri daha iyi bölünebilir hale getirdiğidir. 21'e 5 bir sonraki adımda tek bir özellikle ayrılabilecek gibi görünür. 18'e 33'ü ayırmak için birkaç tur daha gerekebilir.

> **Ders sonrası not:** Hoca A1 true dalının entropisini ilk söylediğinde "0.75" dedi, hemen ardından slayttaki hesabı okurken 0.71 dedi. Doğrusu: $E([21+, 5-]) \approx 0.706$, yani yaklaşık 0.71. Hassas hesapla $Gain(S, A1) \approx 0.266$ ve $Gain(S, A2) \approx 0.121$ çıkar; slayttaki 0.27 ve 0.12 değerleriyle uyumludur.

11. PlayTennis Örneğiyle Adım Adım Ağaç Kurma

Şimdi 14 günlük PlayTennis verisine dönülüyor. Soru şu: Dört özellikten hangisi köke konursa olabildiğince saf bir bölünme elde edilir? Bölünme ne kadar saf olursa ağaç o kadar az uzar.

11.1 Toplam entropi

14 günün 9'unda tenis oynanmış (Yes), 5'inde oynanmamış (No): $[9+, 5-]$.

$$E(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940$$

Örnekler iki sınıfa hemen hemen eşit dağıldığı için entropi 1'e yakındır.

11.2 Örnek 3: Wind'in kazancı

Verilen: Wind iki değer alıyor.

- Weak: 8 gün. 6'sında oynanmış, 2'sinde oynanmamış: $[6+, 2-]$.
- Strong: 6 gün. 3'ünde oynanmış, 3'ünde oynanmamış: $[3+, 3-]$.

Adım 1: Dalların entropisi.

$$E(S_{Weak}) = -\frac{6}{8}\log_2\frac{6}{8} - \frac{2}{8}\log_2\frac{2}{8} = 0.811$$

$$E(S_{Strong}) = -\frac{3}{6}\log_2\frac{3}{6} - \frac{3}{6}\log_2\frac{3}{6} = 1.0$$

Strong dalında iki sınıfta eşit örnek olduğu için entropinin 1 çıktığı gözle de görülür.

Adım 2: Kazanç.

$$Gain(S, Wind) = 0.940 - \frac{8}{14}(0.811) - \frac{6}{14}(1.0) = 0.048$$

Sonuç: Wind köke konursa kazanç yalnızca 0.048 olur.

11.3 Örnek 4: Humidity'nin kazancı

Verilen: Humidity iki değer alıyor.

- High: 7 gün, $[3+, 4-]$.
- Normal: 7 gün, $[6+, 1-]$.

Adım 1: Dalların entropisi.

$$E(S_{High}) = -\frac{3}{7}\log_2\frac{3}{7} - \frac{4}{7}\log_2\frac{4}{7} \approx 0.985$$

$$E(S_{Normal}) = -\frac{6}{7}\log_2\frac{6}{7} - \frac{1}{7}\log_2\frac{1}{7} \approx 0.592$$

High dalında örnekler hemen hemen eşit dağıldığı için entropi yine 1'e yakındır.

Adım 2: Kazanç.

$$Gain(S, Humidity) = 0.940 - \frac{7}{14}(0.985) - \frac{7}{14}(0.592) = 0.151$$

> **Ders sonrası not:** Hoca burada bir an "14 günün 7'sinde rüzgar kuvvetliymiş" dedi, sonra "nem" diye düzeltti. Doğrusu: 7 günde nem yüksek (High), 7 günde normal.

11.4 Örnek 5: Outlook ve Temperature

Outlook üç değer alıyor. Değer sayısının ikiden fazla olması hesapta bir şey değiştirmez; her değer için ayrı entropi hesaplanır.

- Sunny: 5 gün, $[2+, 3-]$, $E \approx 0.971$.
- Overcast: 4 gün, $[4+, 0-]$, $E = 0$.
- Rain: 5 gün, $[3+, 2-]$, $E \approx 0.971$.

Kazanç (ara adım eklendi):

$$Gain(S, Outlook) = 0.940 - \frac{5}{14}(0.971) - \frac{4}{14}(0) - \frac{5}{14}(0.971) \approx 0.247$$

Temperature'ın kazancı altyazıda sözlü olarak verilmiyor. Tablodan hesaplanınca (Hot $[2+, 2-]$, Mild $[4+, 2-]$, Cool $[3+, 1-]$) $Gain(S, Temperature) \approx 0.029$ çıkar.

Karşılaştırma:

Özellik	Kazanç
Outlook	0.247
Humidity	0.151
Wind	0.048
Temperature	0.029

Sonuç: Bütün eğitim örnekleri için en yüksek kazanç Outlook'tadır. **Outlook köke yerleştirilir.**

> **Ders sonrası not:** Hoca Outlook dağılımını önce "5 Sunny, 4 Overcast, 4 Rain" dedi, sonra "5 Rain" diye düzeltti. Doğrusu: 5 Sunny, 4 Overcast, 5 Rain (toplam 14).

11.5 Alt ağaçlar: aynı işlem rekürsif olarak

Kök belirlendikten sonra her dal için aynı işlem, yalnız o dala düşen örneklerle ve henüz kullanılmamış özelliklerle tekrar edilir.

Sunny dalı (5 örnek: günler 1, 2, 8, 9, 11; $[2+, 3-]$, $E \approx 0.971$). Kalan özellikler Humidity, Temperature ve Wind için kazanç hesaplanır:

- $Gain(S_{Sunny}, Humidity) = 0.970$
- $Gain(S_{Sunny}, Temperature) = 0.570$
- $Gain(S_{Sunny}, Wind) = 0.019$

Humidity seçilir. Humidity High ise bütün örnekler No, Normal ise bütün örnekler Yes sınıfındadır. İki dal da saftır; Sunny dalı burada biter. Humidity'nin kazancı (0.970) Sunny dalının entropisine eşittir; bu, Humidity'nin bu 5 örneği tamamen saf gruplara ayırdığını gösterir.

Overcast dalı (4 örnek, hepsi Yes). Bütün örnekler aynı sınıfta. Aşağı devam etmeye gerek yok: Outlook = Overcast ise Yes.

Rain dalı (5 örnek: günler 4, 5, 6, 10, 14; $[3+, 2-]$). Hoca bu dal için hesabın aynı mantıkla yapılacağını söyledi ve yalnız sonucu gösterdi. Hesaplanınca (ara adım eklendi):

- $Gain(S_{Rain}, Wind) \approx 0.971$
- $Gain(S_{Rain}, Humidity) \approx 0.020$
- $Gain(S_{Rain}, Temperature) \approx 0.020$

Wind seçilir. Weak ise bütün örnekler Yes, Strong ise bütün örnekler No.

Son ağaç:

```
Outlook
├─ Sunny    -> Humidity
│              ├─ High   -> No
│              └─ Normal -> Yes
├─ Overcast -> Yes
└─ Rain     -> Wind
               ├─ Weak   -> Yes
               └─ Strong -> No
```

Yapılan işlem özetle şudur: Saf bir duruma ulaşana kadar, kalan örnekler için bilgi kazancı hesaplanır, en yüksek kazançlı özellik seçilir ve örnekler onun değerlerine göre bölünür.

Soru: Outlook seçildikten sonra Sunny ve Rain dallarında ayrı ayrı bilgi kazancı hesaplanınca, iki dalda da en yüksek kazancın Humidity'de çıkması mümkün mü? İkisinde de kullanabilir miyiz? **Cevap:** Evet, mümkün ve kullanılabilir. Dallar birbirinden bağımsız hesaplanır. Hangi dalda hangi özelliğin kazancı en yüksekse o seçilir, başka çare yoktur. Kısıt yalnız şudur: Bir dal boyunca yukarıdan aşağı inerken, o yol üzerinde daha önce seçilmemiş özellikler kullanılır. Örneğin Outlook kullanıldıysa altında Humidity ve Wind kullanılabilir. Humidity'nin altına inildiyse o yolda Humidity'ye bir daha bakılmaz. Her dal kendi içinde değerlendirilir. Hoca böyle durumlarda genellikle ağaç budamaya da gidilebileceğini ekledi.

12. ID3 Algoritması ve Occam'ın Usturası

12.1 ID3

Bu şekilde ağaç kurmak için en çok kullanılan algoritmalardan biri **ID3**'tür. Ross Quinlan tarafından geliştirilmiştir.

Ad: **I**terative **D**ichotomiser **3**.

- **Iterative (yinelemeli):** Aynı işlem tekrar tekrar yapılır. Önce kök bulunur, sonra elde kalan alt ağaç için yeni kök bulunur, sonra onun alt ağacı için, ve böyle devam eder.
- **Dichotomiser (bölücü):** Her adımda eldeki bilgi alt kümelere bölünür. Örneğin Outlook ile veri üç parçaya bölünür, sonra her parça tekrar bölünür.

Algoritma olarak bakılırsa ID3 **yukarıdan aşağı (top-down), açgözlü (greedy)** bir algoritmadır:

- **Top-down:** Ağacı kurmaya kökten başlar, kökten aşağı ilerler.
- **Greedy:** Açgözlü algoritmalar o an için en iyi çözümü seçer. ID3 de her alt ağaç için o an en iyi özelliği bulup düğüm yapar.

  Hocanın vurgusu: En iyi karar ağacını bulmak zor (NP-zor) bir problemdir. ID3'ün bulduğu ağacın en iyisi olmama ihtimali vardır. Olası durumlar içinden, eldeki imkanlarla olabilecek en iyisi bulunmaya çalışılır.

Hoca bunu Örnek 2 üzerinden açıkladı. A1'in kazancı 0.27 olduğu için kökte A1 seçildi. Gözle bakınca da A1'in dalları daha saf, A2'nin true dalı daha karışık görünüyor; A2 seçilirse ağaç daha aşağı uzayabilir gibi duruyor. Bu yüzden o an için en iyi seçim A1'dir. Ama belki de A2 seçilseydi, bir sonraki adımda seçilecek özellik ağacı tek hamlede bitirecekti; A1'in altındaki özellik ise bitirmeyebilir. ID3 bunu bilemez, yalnız o anki durum için en iyi özelliği seçer.

Algoritma:

```
ID3(örnekler, özellikler):
  1. Bütün örnekler aynı sınıftaysa (perfectly classified):
       o sınıfla etiketli bir yaprak döndür.
  2. A <- özellikler içinde bilgi kazancı en yüksek olan.
     A'yı bu alt ağacın kök düğümü yap.
  3. A'nın her v değeri için:
       S_v <- A = v olan örnekler
       bu dalın alt ağacı <- ID3(S_v, özellikler - {A})
  4. Düğümü döndür.
```

Yapraklara ulaşana kadar aynı işlem rekürsif olarak tekrarlanır: en iyi özelliği seç, köke yerleştir, o özelliğin değerlerine göre alt ağaçları oluştur.

Ağaç kurulduktan sonra kurallara çevrilir (bkz. 7.5). Sınıfı bilinmeyen bir örnek bu kurallarla sınıflandırılır. Örneğin Outlook = Sunny ve Humidity = Normal ise tenis oynanacaktır. Outlook = Overcast ise tenis oynanacaktır.

12.2 Occam'ın usturası (Occam's razor)

ID3, Occam'ın usturası denen felsefeye dayanır. Slayttaki ilkenin İngilizcesi:

> "Entities are not to be multiplied beyond necessity."

Hocanın yorumu: En iyi çözüm en basit çözümdür. Birden fazla çözüm aynı sonuca çıkıyorsa, en basit ve en az karmaşık olanı seçin. Karar ağacında bu, aynı başarıyı veren ağaçlardan en kısa olanın tercih edilmesi anlamına gelir. Hoca ilkeyi budama konusunda da "simpler is better" diye tekrar hatırlattı.

13. Aşırı Öğrenme (Overfitting)

13.1 Tanım ve sezgi

Aşırı öğrenme (overfitting), eğitim örneklerini gereğinden fazla öğrenmektir. Türkçede "ezberleme" de denir.

Hoca bunu doğrusal olmayan karar sınırı slaytına dönerek bir örnekle anlattı. Kız ve erkek öğrencileri boy ve kilo özellikleriyle sınıflandırdığımızı düşünelim. Veride 1,90 m boyunda, 80 kg bir kız öğrenci var. Bu örnek, iki özellik uzayında erkeklerin tarafına düşüyor. İki seçenek vardır:

1. **Genel bir sınır:** Model genel olarak iyi sınıflandıran bir karar sınırı çizer. Sınıfların çoğunluğu doğru ayrılır; arada birkaç örnek yanlış sınıflandırılır. Bu kız öğrenci de yanlış sınıfta kalır.
2. **Hırslı bir sınır:** "Bütün eğitim örneklerim doğru sınıflandırılmalı" denir. Bu durumda sınır, o tek örneğin etrafından da dolanır; tek tek kırmızı örnekleri ayırmak için de ayrı sınır parçaları geçer.

Bir öğrenciye bunun iyi mi kötü mü olduğu soruldu. Öğrenci, başka verilerde modelin şaşıracağı için kötü olduğunu söyledi; hoca onayladı. Nedeni şudur: O tek örneğin hemen yakınına sınıfı bilinmeyen yeni bir örnek gelsin. Çevresinde bir sürü diğer sınıftan örnek olduğu için sezgi, bu örneğin o sınıfa ait olduğunu söyler. Ama sınıflandırıcı çok hassas bir sınır çizdiği için örneği tek örneğin bölgesine, yani yanlış sınıfa atar.

Sonuç: Eğitim örnekleri %100 başarıyla sınıflandırılır, ama test örneklerinde (bilinmeyen gerçek dünya örneklerinde) başarı düşer. Hoca overfitting'i ileride ayrı bir derste daha ayrıntılı şekillerle anlatacağını söyledi.

13.2 Overfitting'in nedenleri

En önemli neden **gürültülü (parazitli, noisy) örneklerdir**.

- **Çelişkili örnekler:** İki örneğin bütün özellik değerleri aynı ama sınıfları farklı. Örneğin bir gün hava kötü ve nem yüksekken tenis oynanmış, başka bir gün aynı koşullarda oynanmamış. Bu bir gürültüdür (noise).
- **Aykırı değerler (outlier):** Bir örnek, diğer sınıfın bütün örneklerinin toplandığı bir yere düşmüş (boy-kilo örneğindeki gibi).
- **Hatalı ya da tartışmalı etiketleme:** Hocaya göre en kötü ve çok sık rastlanan durum. Etiketleyen kişi öyle uygun görmüştür. Örneğin geçen derste konuşulan doküman etiketleme probleminde, etiketleyen kişi aslında önemli olan bir dokümanı "önemsiz" diye etiketlemiş olabilir.

13.3 Karar ağacında overfitting

Slaytta PlayTennis verisine gürültülü bir örnek ekleniyor: Outlook = Sunny, Temperature = Hot, Humidity = Normal iken tenis **oynanmamış**. Oysa veride Sunny ve Normal nemli günlerin hepsinde oynanmıştı. Bu örnek eklenince Sunny → Humidity = Normal dalı artık saf olmaz: hem Yes hem No örnekleri vardır. Ağaç orada bitmez, saf olana kadar aşağı doğru uzamaya devam eder. Ağaç, tek bir gürültülü örnek yüzünden büyür.

Slayttaki grafikte yatay eksen ağacın düğüm sayısı, dikey eksen doğruluktur (accuracy, sınıflandırma başarısı):

- **Düz çizgi (eğitim verisi):** Düğüm sayısı arttıkça doğruluk sürekli artar. Düğüm ne kadar çoksa o kadar çok örnek doğru ayrılır.
- **Kesikli çizgi (test verisi):** Doğruluk bir noktaya kadar artar, sonra düşmeye başlar.

Bunun nedeni, düğüm sayısı arttıkça sistemin overfitting'e gitmesidir. Bütün eğitim örnekleri başarıyla sınıflandırılır ama test örneklerinde, az önceki gibi, aslında o sınıfa ait olmayan örnekler de yanlış sınıflandırılmaya başlanır.

> **Hocanın vurgusu:** Karar ağacında ağaç ne kadar büyütülürse overfitting o kadar ortaya çıkar. O halde ağacı çok büyütmemek gerekir.

13.4 Önlem: iki yaklaşım

1. Ağacı kurarken dikkatli olmak, fazla büyütmemek (erken durdurma).
2. Önce ağacı tam olarak kurmak, ardından budamak (pruning).

13.5 Ağacın iyi olup olmadığını anlamak: eğitim, doğrulama, test

Sorun, eğitim verisinde başarı yüksekken test verisinde başarının düşmesiydi. O halde ağacın iyi olup olmadığı, eğitimde kullanılmamış verilerle başarı ölçülerek anlaşılır. Hoca performans ölçümünü 1-2 hafta sonra ayrıca anlatacağını, önce model mantığının anlaşılmasını istediğini söyledi. Ana fikri şimdiden verdi:

Eldeki örnekler üç parçaya ayrılır:

- **Eğitim (train):** Model bu örneklerle eğitilir.
- **Doğrulama (validation):** Başarı bu örneklerle ölçülür. Başarı düşükse model değiştirilir; örneğin karar ağacı budanır.

- **Test:** Model artık oturduğunda, modele hiç dokunmadan, bu bağımsız örneklerle son başarı ölçülür ve "bu modelin başarısı şudur" denir.

Hoca bunu sektördeki deyişle anlattı: Train ile eğitilir, validation ile başarı ölçülüp gerekli iyileştirmeler yapılır, sonra model "canlıya çıkar" ve test edilip başarısı raporlanır.

14. Budama (Pruning)

14.1 Azaltılmış hata budaması (reduced error pruning)

Adımlar:

1. Veri eğitim (train) ve doğrulama (validation) olarak iki parçaya ayrılır.
2. Ağaç eğitim verisiyle tam olarak büyütülür.
3. Ağaç yapraklardan köke doğru budanarak yukarı çıkılır. Bir düğüm budanınca alt ağacı silinir ve düğüm, o noktadaki örneklerin çoğunluk sınıfıyla etiketli bir yaprak olur.
4. Her budamada doğrulama başarısına bakılır. Budama başarıyı düşürmeye başladığı noktada durulur.

Ağacın o kadar aşağı uzamasının nedeni, o noktadaki bütün örneklerin aynı sınıfta olmamasıdır; ağaç, örnekler aynı sınıfa düşene kadar iner. Budama bu fazladan inişi geri alır.

Örnek 6: Bir alt ağacı budamak. Hoca slayttaki bir örneği anlattı.

- Eğitimde ağaç bir noktaya inmiş. O noktada iki negatif, bir pozitif eğitim örneği kaldığı için yeni bir özellik seçilmiş: Color (renk). Bu özellikle örnekler ayrılmış: Color = Red ise pozitif, Color = Blue ise negatif.
- Doğrulama örnekleri bu noktaya gelince şöyle dağılıyor:
  - Color = Red: $[1+, 3-]$. Dal "pozitif" dediği için 3 yanlış sınıflandırma.
  - Color = Blue: $[1+, 1-]$. Dal "negatif" dediği için 1 yanlış sınıflandırma.
  - Toplam: 6 doğrulama örneğinde 4 yanlış sınıflandırma.
- Bu noktadaki 6 doğrulama örneğinin 4'ü negatif, 2'si pozitif. Color düğümü tamamen kaldırılıp dal doğrudan "negatif" yaprağına çevrilirse (ara adım eklendi) yalnız 2 pozitif örnek yanlış sınıflandırılır: 4 yerine 2 hata.
- Budama doğrulama başarısını artırdığı için o alt ağaç budanır.

Slayttaki grafikte (düğüm sayısı - doğruluk), tam büyütülmüş ağaç geriye doğru budandıkça test doğruluğu arttı, arttı ve yaklaşık 20 düğüm budandığında en yüksek değere (yaklaşık 0.73) ulaştı. Test başarısının en yüksek olduğu nokta, budamanın durdurulması gereken yerdir.

14.2 Kural sonrası budama (rule post-pruning)

1. Önce ağaç kurallara çevrilir.
2. Sonra her kuralın koşulları birbirinden bağımsız olarak çıkarılır ve başarı ölçülür.

Slaytta bu yöntem için "en sık kullanılan yöntem" (most frequently used method) yazıyor. Hocaya göre ise zor bir yöntem olduğu için az kullanılıyor; azaltılmış hata budaması daha çok kullanılan yaklaşım.

Hoca budamaya bu derste yalnız giriş yaptığını, uzun süre karşılıklı olmadan dinlemenin yorucu olduğunu bildiği için örnekleri bir sonraki derse bıraktığını söyledi. Gelecek derste budama örnekler üzerinden anlatılacak.

15. Karar Ağaçlarının Güçlü ve Zayıf Yanları

15.1 Güçlü yanlar

- **Basit ve başarılı:** Karar ağaçları hem basit hem başarılı öğrenme modellerinden biridir.
- **Hızlı:** Algoritma derslerinden hatırlanacağı gibi, ağacın bir dalına girildiğinde öbür dallara bir daha bakılmaz. Büyük bir bilgi kümesi hiç kullanılmadan elenmiş olur. Her şeyi sürekli kontrol etmek yerine eleyerek ilerlendiği için, özellikle test aşamasında (bilinmeyen örneğin sınıflandırılmasında) karar ağaçları hızlıdır.
- **Kurala çevrilebilir:** Ağaç kurallara dönüştürülebilir; anlaşılır.
- **Gürültüye karşı çözüm:** Gürültülü veri problemi ağacı budayarak çözülebilir.

- **Nominal özellikler** doğrudan kullanılabilir (bkz. 7.2).

15.2 Zayıf yanlar

- **En iyi ağaç garantisi yok:** "En iyi karar ağacı" diye bir şey bulunamaz; eldeki imkanlarla en iyi ağaç bulunur.
- **Çok özellikte karmaşıklık:** Derste 3-4 özellikle gösterildi. Özellik sayısı 20-30'u bulunca ağaç çok karmaşıklaşır. Anlamak, kurala dönüştürmek, takip etmek zorlaşır.
- **Özellik vektörü baştan bilinmeli:** Özellik vektöründeki özellik sayısının baştan biliniyor olması gerekir. Hoca bazı modellerde bunun böyle olmadığını belirtti.
- **Artımlı (incremental) değil:** Yeni örnekler ya da yeni özellikler geldiğinde mevcut ağaç kullanılamaz, "bunu da ekleyip kullanayım" denemez. Ağacın yeni baştan kurulması gerekir.

Dersin Toparlaması

Bu derste eğitmenli öğrenmenin temel yapısı kuruldu: Etiketli bir veri kümesinden, girişleri çıkışlara en az hatayla eşleyen bir model öğrenilir ve bu model bilinmeyen örneklerin çıkışını tahmin eder. Çıkış sürekliyse problem regresyon, ayrık bir sınıfsa sınıflandırmadır. Her sınıflandırıcı bir karar sınırı çizer; amaç eğitim verisini ezberlemeden genel bir sınır bulmaktır.

İlk model olarak karar ağaçları işlendi. Karar ağacı, eğer-ise kontrolleriyle özellik uzayını saf bölgelere ayırır. Hangi özelliğin köke konacağı entropi ve bilgi kazancıyla seçilir; ID3 bu seçimi her alt ağaç için açgözlü biçimde tekrarlar ve PlayTennis örneğinde Outlook kökte, Humidity ve Wind alt dallarda olan ağacı verir. Ağaç büyüdükçe eğitim başarısı artar ama test başarısı bir noktadan sonra düşer; buna aşırı öğrenme denir. Önlem olarak ağaç ya az büyütülür ya da doğrulama verisiyle budanır. Gelecek derste budama örnekler üzerinden anlatılacak.

Ders Sonrası Notlar

> **Ders sonrası not:** Altyazıda "Rose Queen" olarak geçen ad, ID3'ü geliştiren **Ross Quinlan**'dır.
>
> **Ders sonrası not:** Hoca ID3'teki "3"ün ağaç anlamındaki "tree" kelimesinin okunuşundan geldiğini söyledi. Doğrusu: "3" bir sürüm numarasıdır; ID3, Quinlan'ın "Iterative Dichotomiser" algoritmasının üçüncü sürümüdür.
>
> **Ders sonrası not:** Hoca en iyi ağacı bulmayı "NP problem, yani polinom zamanda çözülemeyen problem" diye tanımladı, sonra "NP-hard" dedi. Doğrusu: En küçük / en iyi karar ağacını bulmak NP-zor (NP-hard) bir problemdir; bilinen polinom zamanlı bir algoritması yoktur. "NP" sınıfının kendisi "polinom zamanda çözülemez" anlamına gelmez; NP, çözümü polinom zamanda doğrulanabilen problemlerdir.
>
> **Ders sonrası not:** Hoca entropi formülünü tanımlarken önce "N tane örnek", sonra "N tane özellik" dedi, ardından kendisi "N tane sınıf" diye düzeltti. Doğrusu: Toplam, sınıflar üzerinden alınır.
>
> **Ders sonrası not:** Hoca bir yerde "entropi, örnekler ne kadar homojen sınıflara dağılmışsa o kadar yüksek olur" dedi. Kastedilen: Örnekler sınıflara ne kadar **eşit** dağılmışsa entropi o kadar yüksektir. Tek bir sınıfta toplanmış (homojen, saf) bir grubun entropisi 0'dır.
>
> **Ders sonrası not:** Hoca Occam'ın usturasını "bilgilere gereğinden fazla önem vermeyin" diye çevirdi. Sözün tam karşılığı: "Varlıklar gereğinden fazla çoğaltılmamalıdır." Yani bir açıklamada gereğinden fazla varsayım ya da öğe kullanılmamalıdır.
>
> **Ders sonrası not:** Altyazıda "Ayata" diye geçen kurum IATA'dır (Uluslararası Hava Taşımacılığı Birliği).

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.